FACULTÉ DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

MATHÉMATIQUES & INFORMATIQUE

Faculté des Sciences Ben M'Sick
Département Mathématiques & Informatique
Master Intelligence Artificielle

Module : Le Cloud Computing & DevOps

Mise en place d'une infrastructure DevOps avec CI/CD et Kubernetes

Sujet : MedDesk - Dossiers Médicaux sur Blockchain

Réalisé par :

Abderrahmane CHAKIR
Aymene CHOUKI
Mohamed BENDIFI

Encadré par :

Pr. Faouzia BENABBOU

Année Universitaire : 2025/2026

Table des matières

I - Code source de l'application et Dockerfile **1**
1 Présentation de l'Application 1
2 Dockerisation du Frontend 2

II - Configuration du pipeline CI/CD **4**
1 Architecture du Pipeline 4
2 Configuration des Secrets GitHub 5
3 Workflow GitHub Actions 5
4 Résultats du Pipeline 6

III - Scripts Terraform et playbooks Ansible **7**
1 Scripts Terraform (Azure) 7
1.1 Résultat de l'exécution Terraform 11
2 Playbooks Ansible 11
2.1 Inventaire Ansible 11
2.2 Playbook de Configuration 12
2.3 Résultat de l'exécution Ansible 13

IV - Manifests Kubernetes (YAML) **14**
1 Deployment 14
2 Service 14
3 Vérification du Déploiement 15

V - Documentation détaillée pour reproduire l'infrastructure **17**
1 Pré-requis du Système 17
1.1 Outils à installer 17
1.2 Accès et Authentification 17
2 Procédure de Reproduction 17
2.1 Phase 1 : Provisionnement avec Terraform 17
2.2 Phase 2 : Configuration avec Ansible 17
2.3 Phase 3 : Configuration du Pipeline GitHub Actions 18
2.4 Phase 4 : Validation du Déploiement 18
3 Dépannage (Troubleshooting) 18

I - Code source de l'application et Dockerfile

Dépôt GitHub : https://github.com/AbdoIR/eHealth

L'application déployée, nommée **MedDesk**, est un système décentralisé de gestion de dossiers médicaux construit sur la blockchain Ethereum. L'objectif de ce projet est de démontrer la capacité de l'infrastructure DevOps à servir une application conteneurisée de manière résiliente et automatisée.

1. Présentation de l'Application

MedDesk est une plateforme orientée santé (E-Health) décentralisée permettant la gestion et la visualisation de dossiers médicaux de manière sécurisée et immuable. L'architecture repose sur :

- **Frontend** : Application React moderne construite avec Vite et la bibliothèque de composants HeroUI, offrant une interface premium avec dark mode et animations fluides (Framer Motion).
- **Blockchain** : Smart contract Solidity (`HealthRecords.sol`) déployé sur Sepolia, assurant un contrôle d'accès basé sur les rôles (Médecins/Patients) et un système de consentement décentralisé.
- **Intégration Web3** : La bibliothèque `ethers.js` connecte le frontend à la blockchain via l'extension MetaMask, permettant des transactions signées et vérifiables.
- **Tests Unitaires** : Utilisation de `Vitest` et `React Testing Library` pour garantir la stabilité des composants et de la logique métier avant chaque déploiement.

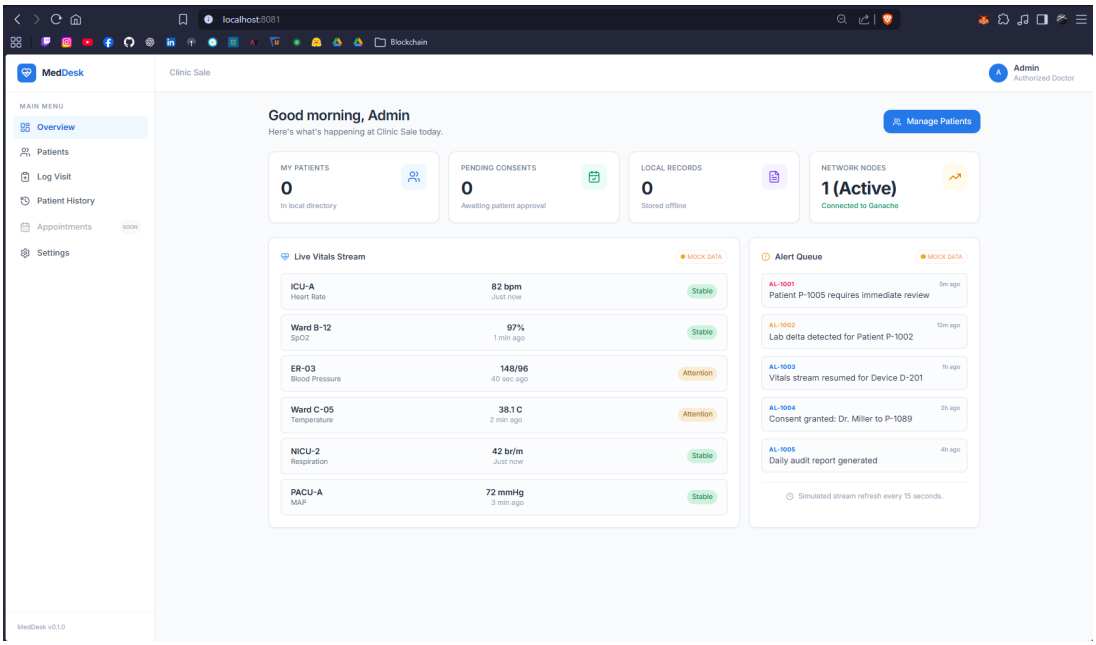

FIGURE 1 – Interface principale de l'application MedDesk

2. Dockerisation du Frontend

Docker est une plateforme permettant de conteneuriser des applications, garantissant un environnement d'exécution identique du développement à la production. Elle repose sur des images légères et portables qui isolent l'application de son infrastructure.

Le Dockerfile utilise une image Node.js Alpine légère pour construire l'application puis la servir via le serveur de prévisualisation intégré de Vite.

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

EXPOSE 4173

CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0"]
```

Listing 1 – Dockerfile - Frontend React

```
[+] Building 79.1s (12/12) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.1s
 => => transferring dockerfile: 211B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                       1.2s
 => [auth] library/node:pull token for registry-1.docker.io                                                             0.0s
 => [internal] load .dockerignore                                                                                       0.0s
 => => transferring context: 189B                                                                                       0.0s
 => [1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9  0.0s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9  0.0s
 => [internal] load build context                                                                                       0.2s
 => => transferring context: 3.03MB                                                                                     0.2s
 => CACHED [2/6] WORKDIR /app                                                                                           0.0s
 => [3/6] COPY package*.json ./                                                                                         0.1s
 => [4/6] RUN npm install                                                                                              26.8s
 => [5/6] COPY . .                                                                                                      0.5s
 => [6/6] RUN npm run build                                                                                            27.7s
 => exporting to image                                                                                                 22.1s
 => => exporting layers                                                                                                14.0s
 => => exporting manifest sha256:bff1eca5e476a409329c4eb9b8cf86cf8989ea7c8465675546969f903fde8d66                       0.1s
 => => exporting config sha256:18b50a79b603f48c6193ff4d196f4f08470f1ff3792aef24c115a287a3d84bc9                         0.1s
 => => exporting attestation manifest sha256:0454f88b9bd250e539430a269d98191d808fed7ba4f7125ff7508329522d3300           0.2s
 => => exporting manifest list sha256:8df12688a8964d712a666064c9f5d888e93d2c98d1534593db6dc0310830bc68                  0.1s
 => => naming to docker.io/library/meddesk:latest                                                                       0.0s
 => => unpacking to docker.io/library/meddesk:latest                                                                    7.5s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/jovodhqbm6c9tn8vtmcj8oo5o
```

Figure 2 – Construction de l'image Docker

```
>> docker start meddesk
meddesk
```

Figure 3 – Exécution du conteneur Docker

```
>> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED        STATUS              PORTS
                    NAMES
a66ff8dab585   abdoir192/meddesk:latest   "docker-entrypoint.s…"   11 hours ago   Up About a minute   0.0.0.0:8081->4173/tcp, [:
:]:8081->4173/tcp   meddesk
```

FIGURE 4 – Affichage des conteneurs Docker

II - Configuration du pipeline CI/CD

Le pipeline CI/CD est orchestré par **GitHub Actions**, un service d'intégration et de déploiement continu intégré directement à GitHub. Ce choix permet d'éviter l'installation d'un serveur CI/CD dédié (comme Jenkins) et de bénéficier de runners hébergés par GitHub, ce qui libère les ressources des VMs cloud pour l'application elle-même.

1. Architecture du Pipeline

Le workflow GitHub Actions se déclenche automatiquement à chaque `push` sur la branche principale. Il exécute les étapes suivantes :

1. **Frontend Tests** : Exécution des tests unitaires et d'intégration via Vitest.
2. **Docker Build & Push** : Construction de l'image Docker (si les tests réussissent).
3. **Déploiement K8s** : Mise à jour du cluster Kubernetes distant.

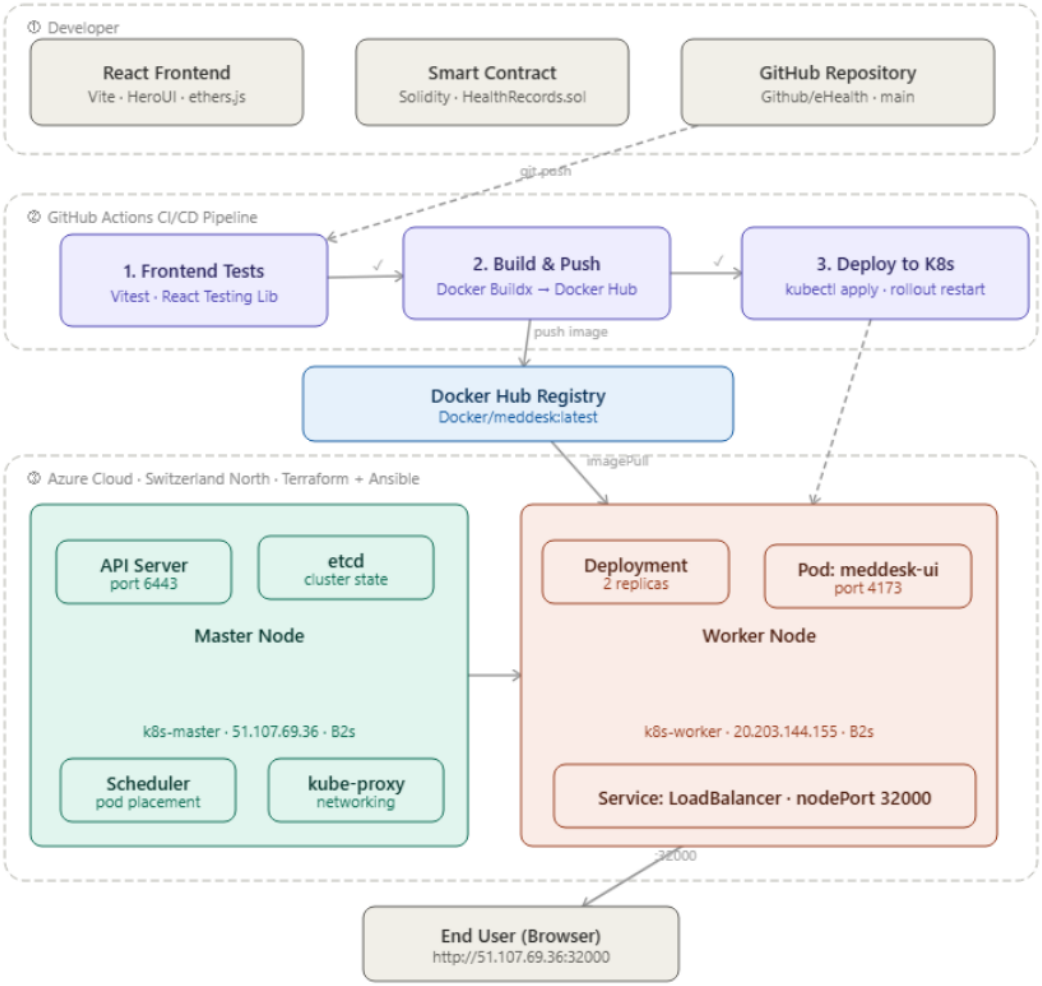

FIGURE 5 – Architecture globale du projet

2. Configuration des Secrets GitHub

Les secrets suivants doivent être configurés dans *Settings* → *Secrets and variables* → *Actions* du dépôt GitHub :

- `DOCKER_USERNAME` : Nom d'utilisateur Docker Hub.
- `DOCKER_PASSWORD` : Mot de passe ou Access Token Docker Hub.
- `KUBE_CONFIG` : Contenu du fichier `kubeconfig` pour l'accès au cluster k3s distant.

3. Workflow GitHub Actions

```yaml
name: Build and Deploy to Kubernetes

on:
  push:
    branches: [ "main" ]

jobs:
  frontend-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
          cache-dependency-path: Frontend/package-lock.json
      - name: Install dependencies
        run: npm install
        working-directory: ./Frontend
      - name: Run Tests
        run: npm test -- --run
        working-directory: ./Frontend

  build-and-push:
    needs: frontend-tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - name: Build and Push Docker Image
        uses: docker/build-push-action@v5
        with:
          context: ./Frontend
```

```yaml
          push: true
          tags: |
            abdoir192/meddesk:latest
            abdoir192/meddesk:${{ github.run_number }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy-to-k8s:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Kubectl
        uses: azure/k8s-set-context@v3
        with:
          method: kubeconfig
          kubeconfig: ${{ secrets.KUBE_CONFIG }}
      - name: Deploy to Cluster
        run: |
          kubectl apply -f k8s/deployment.yml --validate=false
          kubectl apply -f k8s/service.yml --validate=false
          kubectl set image deployment/meddesk-ui meddesk=abdoir192/
             meddesk:${{ github.run_number }}
          kubectl rollout restart deployment/meddesk-ui
```

Listing 2 – .github/workflows/main.yml

4. Résultats du Pipeline

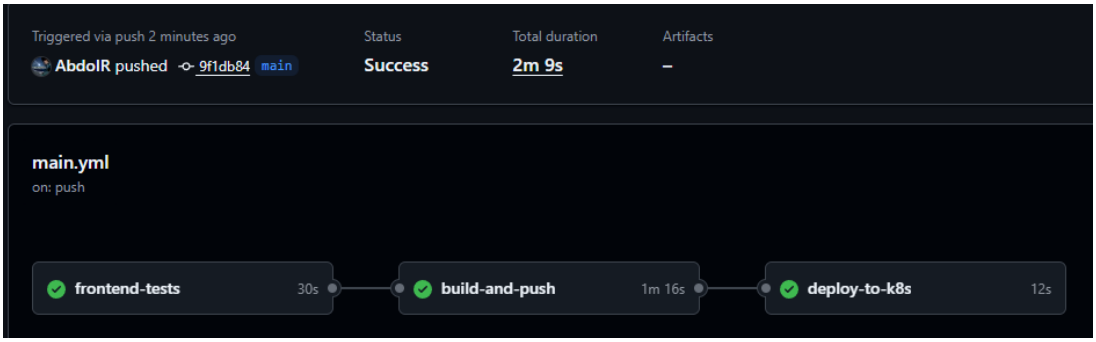

FIGURE 6 – Pipeline GitHub Actions exécuté avec succès

III - Scripts Terraform et playbooks Ansible

L'infrastructure est déployée via des scripts Terraform (Infrastructure as Code) et configurée via Ansible.

1. Scripts Terraform (Azure)

Le script suivant permet de provisionner deux machines Standard_B2s, incluant les règles de pare-feu (NSG) nécessaires pour Kubernetes.

```
# 1. Define Azure Provider
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = ">= 3.7.0"
    }
  }
}

provider "azurerm" {
  features {
    resource_group {
      prevent_deletion_if_contains_resources = false
    }
  }
  resource_provider_registrations = "none"
}

# 2. Create a Resource Group
resource "azurerm_resource_group" "rg" {
  name     = "eHealth-Cloud-RG"
  location = "switzerlandnorth"
}

# 3. Networking: Virtual Network and Subnet
resource "azurerm_virtual_network" "vnet" {
  name                = "k8s-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name                 = "k8s-subnet"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}

# 4. Public IPs for SSH and Web Access
resource "azurerm_public_ip" "master_ip" {
```

```
  name                = "master-pip"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

resource "azurerm_public_ip" "worker_ip" {
  name                = "worker-pip"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

# 5. Network Security Group (Firewall)
resource "azurerm_network_security_group" "k8s_nsg" {
  name                = "k8s-nsg"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  # Allow SSH for Ansible Management
  security_rule {
    name                       = "SSH"
    priority                   = 1001
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  # Allow k3s API Server (Communication between nodes)
  security_rule {
    name                       = "k3s-API"
    priority                   = 1002
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "6443"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  # Allow Web Application Traffic
  security_rule {
    name                       = "K8s-NodePorts"
    priority                   = 1003
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "30000-32767"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
```

```
  }
}

# 6. Network Interfaces (NICs) with NSG Associations
resource "azurerm_network_interface" "master_nic" {
  name                = "master-nic"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.master_ip.id
  }
}

resource "azurerm_network_interface" "worker_nic" {
  name                = "worker-nic"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.worker_ip.id
  }
}

resource "azurerm_network_interface_security_group_association" "
    master_assoc" {
  network_interface_id      = azurerm_network_interface.master_nic.id
  network_security_group_id = azurerm_network_security_group.k8s_nsg.
      id
}

resource "azurerm_network_interface_security_group_association" "
    worker_assoc" {
  network_interface_id      = azurerm_network_interface.worker_nic.id
  network_security_group_id = azurerm_network_security_group.k8s_nsg.
      id
}

# 7. Virtual Machines
resource "azurerm_linux_virtual_machine" "master" {
  name                  = "k8s-master"
  resource_group_name   = azurerm_resource_group.rg.name
  location              = azurerm_resource_group.rg.location
  size                  = "Standard_B2s"
  admin_username        = "azureuser"
  network_interface_ids = [azurerm_network_interface.master_nic.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("C:/Users/abdoi/.ssh/id_rsa.pub")
  }

```

```
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}

resource "azurerm_linux_virtual_machine" "worker" {
  name                  = "k8s-worker"
  resource_group_name   = azurerm_resource_group.rg.name
  location              = azurerm_resource_group.rg.location
  size                  = "Standard_B2s"
  admin_username        = "azureuser"
  network_interface_ids = [azurerm_network_interface.worker_nic.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("C:/Users/abdoi/.ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}

# 8. Outputs for Ansible Inventory
output "master_public_ip" {
  value = azurerm_public_ip.master_ip.ip_address
}

output "worker_public_ip" {
  value = azurerm_public_ip.worker_ip.ip_address
}
```

Listing 3 – main.tf - Provisionnement Azure

1.1. Résultat de l'exécution Terraform

```
Outputs:

master_public_ip = "51.107.69.36"
worker_public_ip = "20.203.144.155"
```

FIGURE 7 – Application du plan Terraform

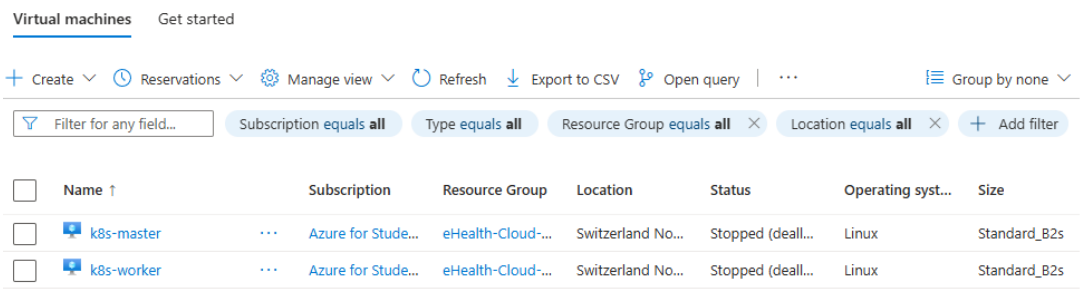

FIGURE 8 – Ressources visibles sur le portail Azure

2. Playbooks Ansible

Ansible est utilisé pour configurer l'environnement Docker et initialiser le cluster k3s sur les instances provisionnées.

2.1. Inventaire Ansible

```
[master]
k8s-master ansible_host=51.107.69.36

[worker]
k8s-worker ansible_host=20.203.144.155

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Listing 4 – inventory.ini - Inventaire des noeuds

2.2. Playbook de Configuration

```yaml
---
- name: Configuration de base et Docker
  hosts: all
  become: yes
  tasks:
    - name: Mise    jour des paquets
      apt: update_cache=yes upgrade=dist

    - name: Installation de Docker
      apt:
        name: docker.io
        state: present

- name: Installation du Master Node
  hosts: master
  become: yes
  tasks:
    - name: Installer k3s sur le Master
      shell: curl -sfL https://get.k3s.io | sh -
      register: k3s_install

    - name: R cup rer le Node Token
      shell: cat /var/lib/rancher/k3s/server/node-token
      register: node_token

- name: Installation du Worker Node
  hosts: worker
  become: yes
  vars:
    master_ip: "{{ hostvars['k8s-master']['ansible_host'] }}"
    token: "{{ hostvars['k8s-master']['node_token']['stdout'] }}"
  tasks:
    - name: Joindre le cluster k3s
      shell: "curl -sfL https://get.k3s.io | K3S_URL=https://{{
         master_ip }}:6443 K3S_TOKEN={{ token }} sh -"
```

Listing 5 – setup-k3s.yml - Configuration du cluster

2.3. Résultat de l'exécution Ansible

```
PLAY [Configuration de base et Docker] ***********************************************************************

TASK [Gathering Facts] ***************************************************************************************
ok: [k8s-master]
ok: [k8s-worker]

TASK [Mise à jour des paquets] *******************************************************************************
TASK [Gathering Facts] ***************************************************************************************
ok: [k8s-master]

TASK [Installer k3s sur le Master] ***************************************************************************
changed: [k8s-master]

TASK [Récupérer le Node Token] *******************************************************************************
changed: [k8s-master]

PLAY [Installation du Worker Node] ***************************************************************************

TASK [Gathering Facts] ***************************************************************************************
ok: [k8s-worker]

TASK [Joindre le cluster k3s] ********************************************************************************
changed: [k8s-worker]

PLAY RECAP ***************************************************************************************************
k8s-master                 : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
k8s-worker                 : ok=5    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

FIGURE 9 – Résultat de l'exécution du playbook Ansible

IV - Manifests Kubernetes (YAML)

Le déploiement utilise des manifests YAML pour définir l'état souhaité de l'application sur le cluster k3s.

1. Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: meddesk-ui
spec:
  replicas: 2
  selector:
    matchLabels:
      app: meddesk
  template:
    metadata:
      labels:
        app: meddesk
    spec:
      containers:
        - name: meddesk
          image: abdoir192/meddesk:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 4173
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

Listing 6 – deployment.yml

Le manifest définit un déploiement de 2 réplicas avec la politique `imagePullPolicy: Always` pour s'assurer que chaque redémarrage utilise la dernière image depuis Docker Hub. Des limites de ressources sont configurées pour éviter qu'un conteneur ne consomme toute la mémoire ou le CPU du noeud.

2. Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: meddesk-service
spec:
  selector:
```

```yaml
    app: meddesk
  ports:
    - protocol: TCP
      port: 80
      targetPort: 4173
      nodePort: 32000
  type: LoadBalancer
```

Listing 7 – service.yml

Le service de type `LoadBalancer` expose l'application sur le port 80, redirigeant le trafic vers le port 4173 du conteneur. Le `nodePort: 32000` permet un accès direct via l'IP publique du noeud sur ce port.

3. Vérification du Déploiement

```
NAME                          READY   STATUS    RESTARTS        AGE
meddesk-ui-79994d7d55-7m85n   1/1     Running   2 (3m31s ago)   41m
meddesk-ui-79994d7d55-h7tgb   1/1     Running   2 (3m49s ago)   41m
```

FIGURE 10 – État des pods sur le cluster

```
NAME             TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes       ClusterIP      10.43.0.1      <none>        443/TCP        3h24m
meddesk-service  LoadBalancer   10.43.112.71   <pending>     80:32000/TCP   31m
```

FIGURE 11 – Services Kubernetes actifs

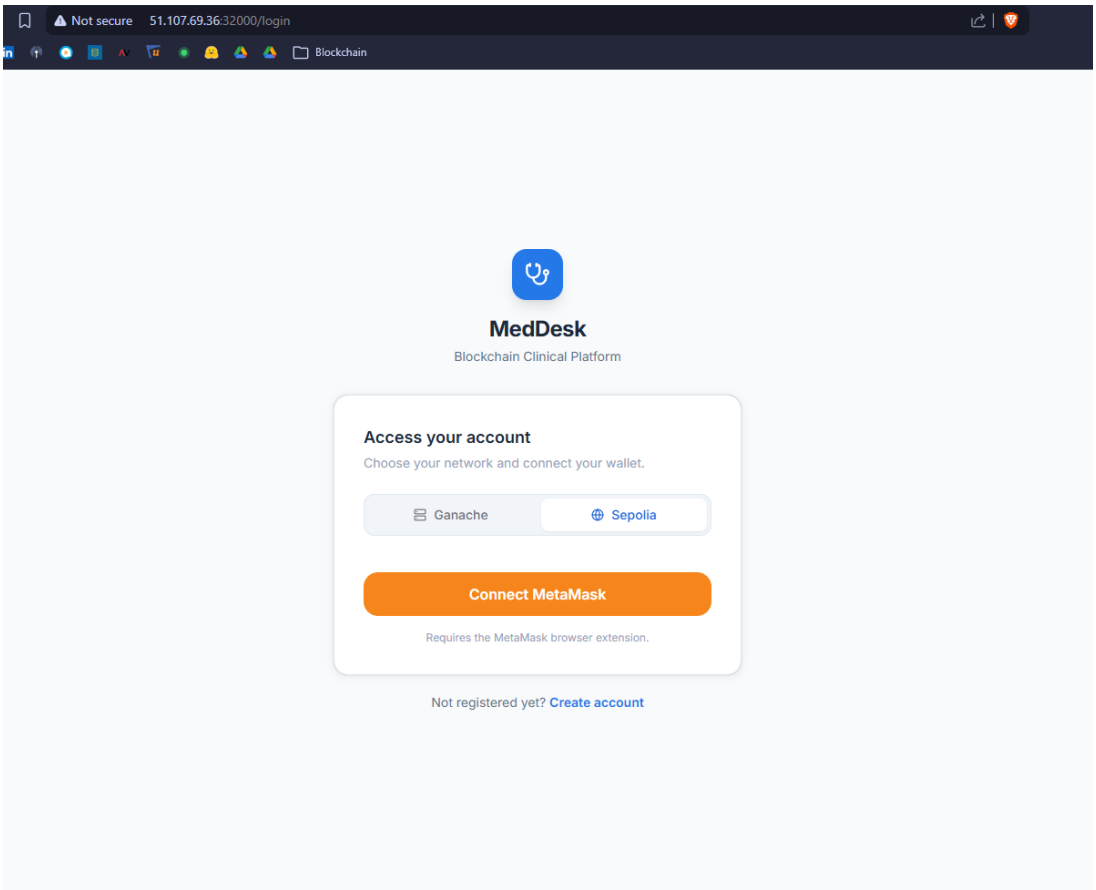

FIGURE 12 – Application déployée et fonctionnelle sur le cluster

V - Documentation détaillée pour reproduire l'infrastructure

1. Pré-requis du Système

1.1. Outils à installer

- **Terraform** (version 1.5+) : Provisionnement de l'infrastructure cloud.
- **Ansible** (version 2.10+) : Configuration automatisée des serveurs.
- **Kubectl** : Interaction avec le cluster Kubernetes (k3s).
- **Azure CLI** : Authentification et gestion des ressources Azure.

1.2. Accès et Authentification

- **Cloud** : Un abonnement Azure actif (possible etudiant).
- **Clés SSH** : Une paire de clés SSH générée et configurée. S'assurer des permissions correctes :

```
chmod 600 ~/.ssh/id_rsa
chmod 644 ~/.ssh/id_rsa.pub
```

Listing 8 – Configuration des permissions SSH

2. Procédure de Reproduction

2.1. Phase 1 : Provisionnement avec Terraform

1. Cloner le repository du projet : `git clone https://github.com/AbdoIR/eHealth.git`
2. Naviguer vers le dossier `terraform/`.
3. Exécuter `terraform init` pour initialiser les providers.
4. Exécuter `terraform plan` pour vérifier les ressources à créer.
5. Exécuter `terraform apply` pour provisionner l'infrastructure sur Azure.
6. Noter les IPs publiques affichées en sortie (outputs).

2.2. Phase 2 : Configuration avec Ansible

1. Mettre à jour l'inventaire Ansible (`inventory.ini`) avec les IPs publiques fournies par Terraform.
2. Vérifier la connectivité SSH : `ansible all -i inventory.ini -m ping`.
3. Lancer le playbook de configuration du cluster :

```
ansible-playbook -i inventory.ini setup-k3s.yml
```

4. Vérifier que le cluster est opérationnel en se connectant au Master :

```
ssh azureuser@<master_ip> "sudo kubectl get nodes"
```

2.3. Phase 3 : Configuration du Pipeline GitHub Actions

1. Configurer les secrets dans le dépôt GitHub (*Settings* → *Secrets*) :
   - `DOCKER_USERNAME` et `DOCKER_PASSWORD` pour Docker Hub.
   - `KUBE_CONFIG` pour l'accès au cluster k3s distant.
2. Créer le fichier `.github/workflows/main.yml` dans le dépôt (voir Chapitre II).
3. Pousser le code sur la branche `main` pour déclencher le pipeline.
4. Vérifier que le job `frontend-tests` s'exécute avec succès avant le déploiement.

2.4. Phase 4 : Validation du Déploiement

1. Se connecter au Master Node via SSH.
2. Exécuter `kubectl get nodes` pour vérifier l'état du cluster.
3. Exécuter `kubectl get pods` pour confirmer le déploiement des réplicas de l'application.
4. Accéder à l'application via le navigateur en utilisant l'IP publique du noeud sur le port exposé par le service Kubernetes.

3. Dépannage (Troubleshooting)

En cas de problème lors de la reproduction, voici les erreurs les plus fréquentes et leurs solutions :

Problème / Erreur	Cause Racine	Solution Appliquée
`Permissions too open` (SSH)	Clé privée `.pem` ayant des permissions trop larges sur l'hôte.	Restreindre les permissions via `chmod 600` ou via les ACLs NTFS sur Windows.
`Connection refused` (Port 6443)	Service `k3s` arrêté sur le Master ou port bloqué au niveau du NSG Azure.	Démarrage du service (`systemctl start k3s`) et ouverture du port 6443 dans le NSG.
`x509: certificate signed by unknown authority`	L'API Server ne reconnaît pas l'IP publique comme valide pour le certificat TLS.	Réinstallation de k3s avec le flag `-tls-san=<IP_Publique>` ou modification de la config.
`Connection refused` (Port 32372/32000)	Mismatch entre le port d'écoute de l'application (Vite : 4173) et le `targetPort` du Service.	Alignement du `targetPort` sur 4173 dans le manifest `service.yml`.
Blocage SSH/Web post-redémarrage	Activation automatique du firewall interne (`ufw`) bloquant les flux entrants.	Désactivation de `ufw` ou ajout de règles persistantes (`ufw allow 22, 6443, 32000`).
Instabilité de l'URL d'accès	Assignation dynamique d'un `nodePort` aléatoire par Kubernetes à chaque déploiement.	Fixation statique du `nodePort` à 32000 et déploiement via `kubectl apply -force`.
Dépendance locale (ngrok)	Utilisation de Jenkins en local imposant une mise à jour manuelle des Webhooks GitHub.	Migration vers **GitHub Actions** pour une CI/CD native et déportée sur le Cloud.
Échec du déploiement (OpenAPI)	Timeout lors du téléchargement des schémas OpenAPI par `kubectl` depuis le cluster.	Utilisation du flag `-validate=false` pour ignorer la validation réseau non-critique.
Échec des tests (Vitest)	Erreur de régression dans le frontend ou environnement de test mal configuré.	Analyse des logs GitHub Actions et correction avant la construction de l'image Docker.

Table 1 – Synthèse des incidents techniques et résolutions